Entity Relationship modeling from an ORM perspective: Part 1

by Dr. Terry Halpin

Director of Database Strategy, Visio Corporation

This is a revised version of a paper that first appeared in the December 1999 issue of the Journal of Conceptual Modeling*, published by InConcept.*

This paper is the first in a series of articles examining data modeling in the Entity Relationship (ER) approach from the perspective of Object Role Modeling (ORM). This article examines basic aspects of the Barker notation for ER.

Introduction

Entity Relationship modeling (ER) views the application domain in terms of entities that have attributes and participate in relationships. For example, the fact that an employee was born on a date is modeled by assigning a birthdate attribute to the Employee entity type, whereas the fact that an employee works for a department is modeled as a relationship between them. This view of the world is quite intuitive, and in spite of the recent rise of UML for modeling object-oriented applications, ER is still the most popular data modeling approach for database applications.

The ER approach was originally proposed by Peter Chen in 1976, in the very first issue of an influential ACM journal [2]. As shown in Figure 1, Chen's original notation used rectangles for entity types, diamonds for relationships, and ellipses for attributes. The double ellipse indicates unique identifier attributes, and the "n and "1" indicate the relationship is many to one (each employee works for at most one department, but many employees may work for the same department).

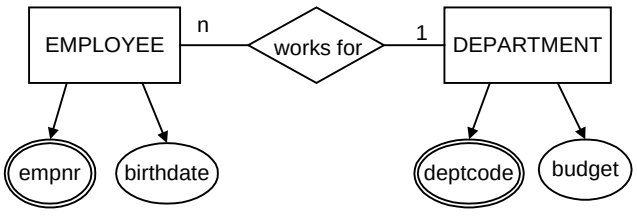

Figure 1 An early ER notation used by Chen

The direction in which relationship names are to be read is formally undecided, unless we add some additional marks (e.g. arrows) or rules (e.g. always read from left to right and from top to bottom). For example, does the employee work for the department, or does the department work for the employee? Although we can use our background knowledge to informally disambiguate this example, it is quite common nowadays to see ER models with relationships whose intended direction can only be guessed at by anybody other than the model's creator. For example, consider the impact of misreading the intended direction for the following: Person killed Animal; Person is loved by Person. This problem is exacerbated if the verb phrase used to name the relationship is shortened to one word (e.g. "work", "love"), unfortunately still a fairly common practice.

Chen's notation evolved over time. His current ER-Designer tool uses hexagons instead of diamonds, and supports n-ary relationships. Outside academia, Chen's notation seems to be rarely used nowadays, so I'll say no more about it here. One of the problems with the ER approach is that there are so many versions of it, with no single standard. In industrial practice, the most popular versions of ER are the Barker and Information Engineering (IE) notations. Another popular data modeling notation is IDEF1X, but since this is a hybrid of ER and relational notation, I don't count it as a true ER representative. As discussed elsewhere [4], UML class diagrams can be regarded as an extended version of ER. The rest of this article focuses on basic aspects of the Barker notation for ER. Later articles will examine IE and IDEF1X.

Barker ER: the basics

I use the term "Barker ER" for the ER notation discussed in the classic treatment by Richard Barker [1]. While Oracle Corporation has long used this notation in its CASE tools, Oracle's Object Designer tool now supports UML as an alternative to its traditional ER notation. For database applications, many modelers still prefer the Barker ER notation in preference to UML, and it will be interesting to see whether this changes over time. Dave Hay, an experienced modeler and ardent fan of the Barker ER notation, argues that "there is no such thing as 'object-oriented analysis'" [6], only object-oriented design, and that "UML is … not suitable for analyzing business requirements in cooperation with business people"[7].

While I agree with Dave Hay that UML class diagrams are less than ideal for data modeling, I feel that his preferred ER notation shares some of UML's weaknesses in being attribute-based. As I've discussed before in a UML context [4, 5], using attributes in a base conceptual model adds complexity and instability, while making it harder to validate models with domain experts using verbalization and sample populations. Attributes are great for logical design, since they allow compact diagrams that directly represent the data structures (e.g. relations or object-relations) used for the actual design. However when I'm performing conceptual analysis, I just want to know what the *facts* and *rules* are about the business, and I want to communicate this information in sentences, so that the

model can be understood by the domain experts. I sure don't want to bother about how facts are grouped into multi-fact structures. Whether some fact will end up in the design as an attribute is not a conceptual issue to me. As Ron Ross says, "Sponsors of business rule projects must sign off on the sentences—not on graphical data models. Most methodologies and CASE tools have this more or less backwards" [7, p.15]. The ORM reporting facilities in Visio Enterprise allow the domain expert to inspect ORM models fully verbalized into sentences with examples, making validation much easier and safer.

Now that I've stated my bias up front, let's examine the Barker ER notation itself. The basic conventions are illustrated in Figure 2. *Entity types* are shown as soft rectangles (rounded corners) with their name in capitals. *Attributes* are written below the entity type name. Some constraint information may appear before an attribute name. A "#" indicates that the attribute is the primary identifier of the entity type, or at least a component of its primary identification scheme. A "*" or heavy dot "•" indicates the attribute is mandatory (i.e. each instance in the database population of the entity type must have a non-null value recorded for this attribute). A "°" indicates the attribute is optional. Some modelers also use a period "." to indicate the attribute is not part of the identifier.

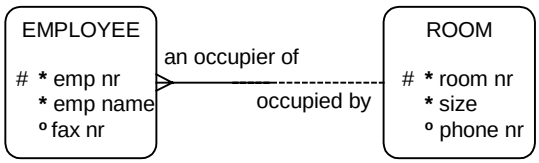

Figure 2 The basic Barker ER notation

Relationships are restricted to binaries (no unaries, ternaries or longer relationships), and are shown as lines with a relationship name at the end from which that relationship name is to be read. This name placement overcomes the ambiguous direction problem mentioned earlier. Both forward and inverse readings may be displayed for a binary relationship, one on either side of the line. This makes the Barker notation superior to UML for verbalizing relationships.

From an ORM perspective, each end (or half) of a relationship line corresponds to a role. Like ORM, Barker ER treats role *optionality* and *cardinality* as distinct, orthogonal concepts, instead of lumping them together into a single concept (e.g. multiplicity in UML). A solid line-half denotes a mandatory role, and a dotted line-half indicates an optional role. For cardinality, a crow's foot intuitively indicates "many", by its many "toes". The absence of a crow's foot intuitively indicates "one". The crow's foot notation was invented by Gordon Everest, who originally used the term "inverted arrow" [3] but now calls it a "fork". Figure 3 shows the correspondence with the ORM notation for uniqueness and mandatory role constraints.

To enable the optionality and cardinality settings to be verbalized, Barker [1, p. 3-5] recommends the following *naming discipline for relationships*. Let *A R B* denote an infix relationship *R* from entity type *A* to entity type *B*. Name *R* in such a way that each of the following four patterns results in an English sentence:

each *A* (**must** | **may**) **be** *R* (**one and only one** *B* | **one or more** *B-plural-form*)

Use “must” or “may” when the first role is mandatory or optional respectively. Use “one and only one” or “one or more” when the cardinality on the second role is one or many respectively. For example, the optionality/cardinality settings in Figure 3(a) verbalize as: each Employee must be an occupier of one and only one Room; each Room may be occupied by one or more Employees. This verbalization convention is good for basic mandatory and uniqueness constraints on infix binaries. However it is far less general than ORM’s approach, which applies to instances as well as types, for predicates of any arity, and covers many more kinds of constraint, with no need for pluralization. As a trivial example, the fact instance “Employee ‘101’ an occupier of Room 23” is not proper English, but “Employee ‘101’ occupies Room 23” is good English.

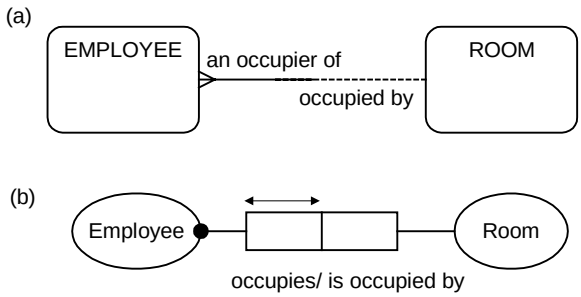

Figure 3 The ER diagram (a) is equivalent to the ORM diagram (b)

If each of the two roles in a binary association may be assigned one of optional/mandatory and one of many/one, there are sixteen patterns. The equivalent Barker ER and ORM diagrams for the first eight of these cases are shown in Figure 4.

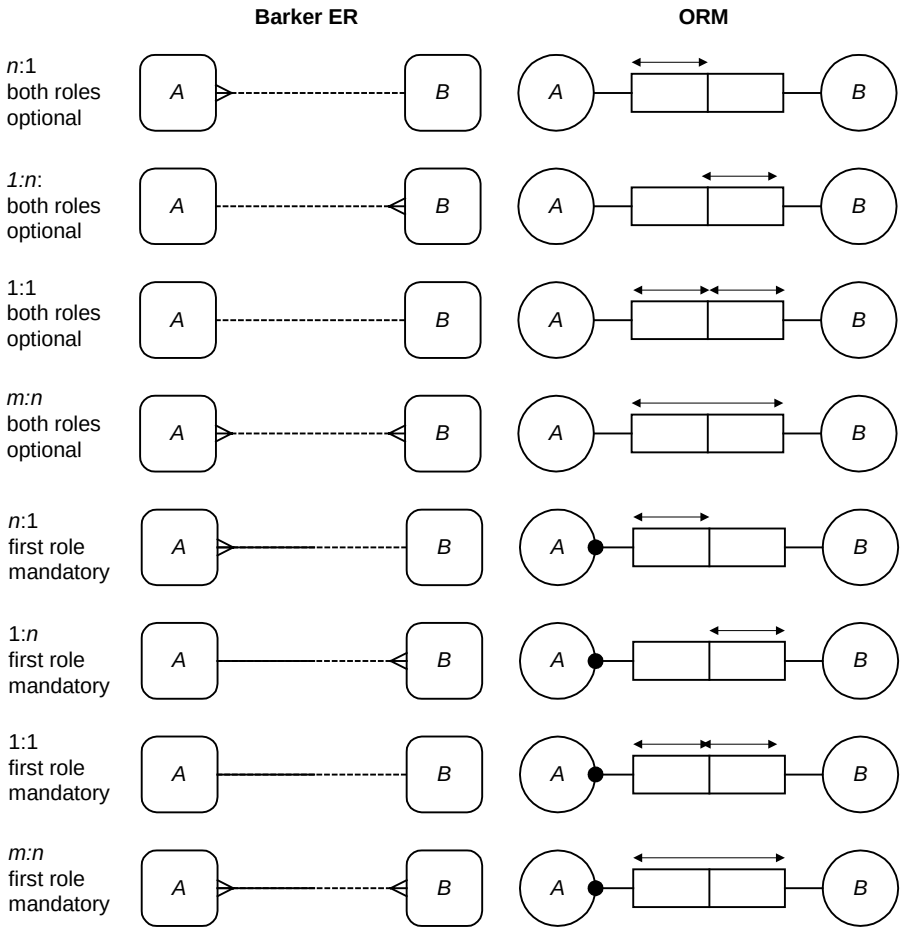

Figure 4 Some equivalent cases

The other eight cases are shown in Figure 5. Although all eight are legal in ORM, the last case where both roles of a many:many relationship are mandatory is considered illegal by Barker.

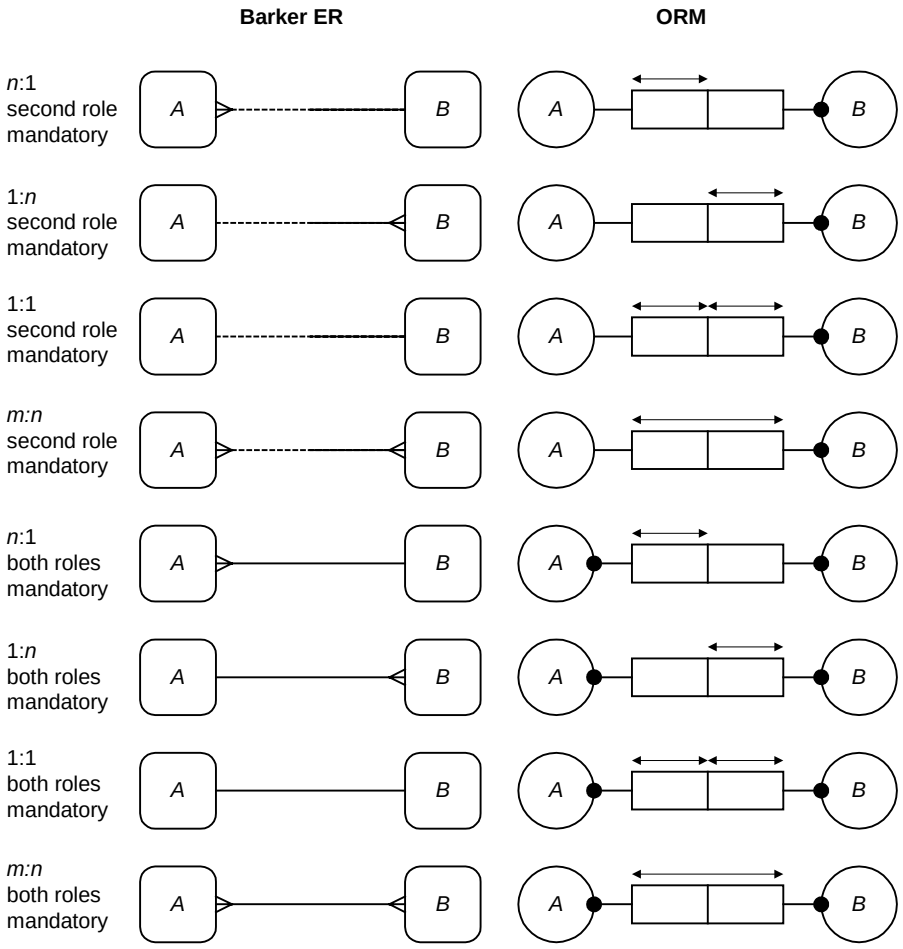

Figure 5 Other equivalent cases

Ring associations that considered illegal by Barker are shown in Figure 6(a). Although rare, they sometimes occur in reality, so should be allowed at the conceptual level, as permitted in ORM. As an exercise, you may wish to invent satisfying populations for the ORM associations in Figure 6 (b). Although considered illegal by Barker, at least some of these patterns are allowed in Oracle's CASE tools.

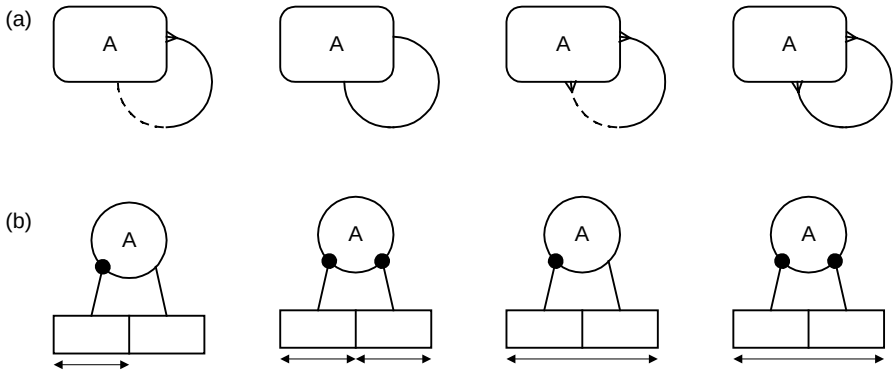

Figure 6 Illegal ring associations in Barker ER (a) that are rare but allowed in ORM (b)

In Barker ER, a bar "|" across one end of a relationship indicates that the relationship is a component of the primary identifier for the entity type at that end. In Figure 7 for example, Employee and Building have simple identifiers, but Room has a composite reference scheme, being identified partly by its room number and partly by the building in which it is included.

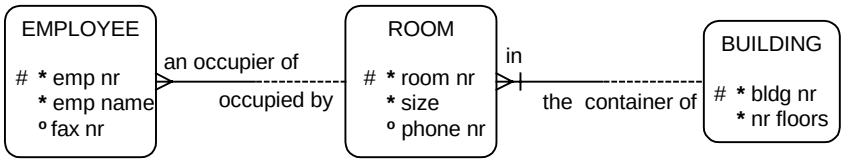

Figure 7 Room is identified by combining its room nr and its relationship to Building

The use of identification bars provides some of the functionality afforded by external uniqueness constraints in ORM. For example, the schemas in Figure 8 are equivalent. The other attributes of Room and Building in ORM would be modeled in ORM as relationships. ORM's external uniqueness notation seems to me to convey more intuitively the idea that each RoomNr, Building combination is unique (i.e. refers to at most one room). But maybe I'm biased. At any rate, this constraint (as well as any other graphic constraint) can be automatically verbalized in natural language.

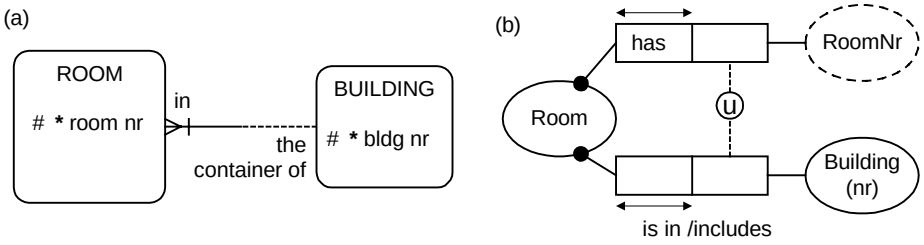

Figure 8 Composite identification in Barker ER (a) and ORM (b)

Some people misread the bar notation for composite identification as a "1", since this is what the symbol means in many other ER notations. But this isn't a problem if you don't have to work with multiple versions of ER. The main problem with the "#" and bar notations is that they cannot be used to declare uniqueness constraints that are not used for a primary identification scheme. A second problem is that they are two very different notations for the same fundamental concept (uniqueness). Because ORM allows constraints to be used wherever they make sense, and always uses relationships instead of attributes, it doesn't have these problems. An example may help illustrate some of these ideas. Suppose we wanted to model the information shown in Table 1, as well as other facts about rooms.

Table 1 A simple data use case for room scheduling

Room	*Time*	*ActivityCode*	*ActivityName*
20	Mon 9 am	VMC	VisioModeler class
20	Tue 2 pm	VMC	VisioModeler class
33	Mon 9 am	AQD	ActiveQuery demo
33	Fri 5 pm	SP	Staff party
…	…	…	…

The table suggests that rooms can be simply identified by room numbers, so let's accept that. One way of modeling the situation in Barker ER is shown in Figure 9. Here the bar notation is used to show that RoomTimeSlot is identified by combining its time and room number.

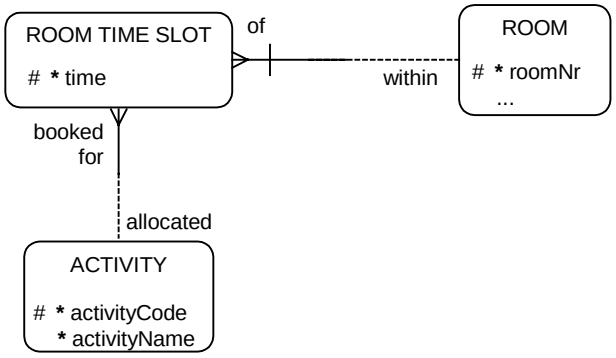

Figure 9 An ER diagram for room scheduling

The use of attributes in this model makes it hard to verbalize and populate the schema for validation purposes. Moreover, there is at least one constraint missing. Compare this with the populated ORM model for the same situation (Figure 10). Here the facts are naturally verbalized as a ternary (Room at Time is booked for Activity) and a binary (Activity has ActivityName). The associated fact tables include the original facts, as well as counter-facts (italicized) to test the constraints. The first counter-row (20, Mon 9 am, AQD) tests the uniqueness constraint that a room at a time is booked for at most one activity. The second counter-row tests the uniqueness constraint that at most one room can be booked for a given activity at a given time. This constraint may well be wrong, but

at least we can express it and test it in ORM. With the ER model there is no way of even specifying the constraint, much less testing it.

The counter rows (SP, Sales phonecalls) and (PTY, Staff party) are designed to check the uniqueness constraints that each Activity has at most one Activity name and vice versa. If these are rejected, the association really is 1:1, as its basic population suggests. Since the ER notation being discussed doesn't include a way of indicating that attributes other than the primary identifier are unique, it isn't very helpful here. As a small point, the Y2K row has been added to the original population to indicate that it is possible for some listed activities to be unscheduled.

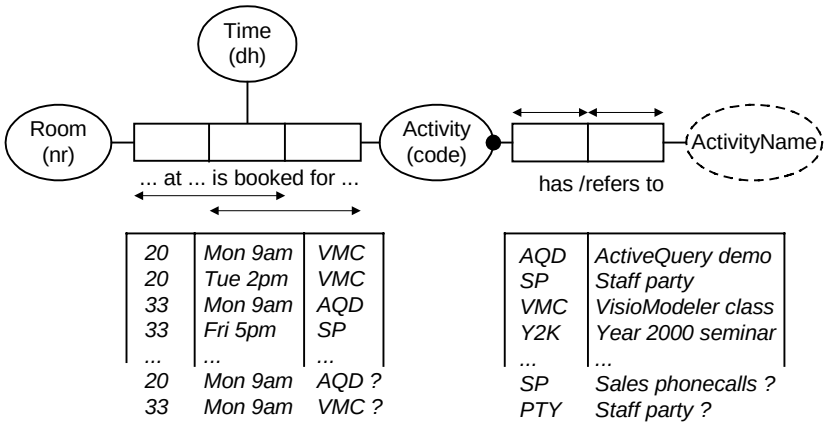

Figure 10 An ORM diagram for room scheduling, with sample and counter data

In case it looks like I'm just bashing attribute-based approaches like ER in this article, let me say again that I find attribute-based models useful for compact overviews and for getting closer to the implementation model. However I generate these by mapping from ORM, which I use exclusively for conceptual analysis. This makes it easier to get the model right in the first place, and to modify it as the underlying domain evolves. Unlike ER (and UML for that matter), ORM was built from a linguistic basis, and its graphic notation was carefully chosen to exploit the potential of sample populations. To reap the benefits of verbalization and population for communication with and validation by domain experts, it's better to use a language that was designed with this in mind. An added benefit of ORM is that its graphic notation can capture many more business rules than popular ER notations.

Next issues

Later articles in this series will consider more advanced aspects of the Barker ER notation, including exclusion constraints, frequency constraints, subtyping and non-transferable relationships, and then examine the Information Engineering notation for ER, before concluding with a discussion of IDEF1X.

References

1. Barker, R. 1990, *CASE*Method: Tasks and Deliverables*, Addison-Wesley, Wokingham, England.

2. Chen, P.P. 1976, 'The entity-relationship model—towards a unified view of data', *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36.

3. Everest, G. 1976, 'Basic Data Structure Models Explained with a Common Example', *Proc. Fifth Texas Conference on Computing Systems*, (Austin, TX, 1976 October 18-19), IEEE Computer Society publications office, Long Beach, CA, pp. 39-45.

4. Halpin, T.A. 1998-9, 'UML data models from an ORM perspective: Parts 1-10', *Journal of Conceptual Modeling*, InConcept, Minneapolis USA.

5. Halpin, T.A. & Bloesch, A.C. 1999, 'Data modeling in UML and ORM: a comparison', *Journal of Database Management*, vol. 10, no. 4, Idea group Publishing Company, Hershey, USA, pp. 4-13.

6. Hay, D.C. 1999, 'There is no object-oriented analysis', *DataToKnowledge Newsletter*, vol. 27, no. 1, Business Rule Solutions, Inc., Houston TX, USA.

7. Hay, D.C. 1999, 'Object orientation and information engineering: UML', *The Data Administration Newsletter*, no. 9, (June 1999), ed. R.S. Reiner, available online at www.tdan.com.

8. Ross, R.G. 1998, *Business Rule Concepts*, Business Rule Solutions, Inc., Houston TX, USA.

Entity Relationship modeling from an ORM perspective: Part 2

Terry Halpin
Microsoft Corporation

Introduction

This article is the second in a series of articles dealing with Entity Relationship (ER) modeling from the perspective of Object Role Modeling (ORM). Part 1 provided a brief overview of the ER approach, and then covered the basics of the Barker ER notation [1], that has long been supported in CASE tools from vendors such as Oracle Corporation. In this notation, entity types are depicted as named, soft rectangles, and binary relationships are shown as lines with forward and inverse names. If shown, attributes are listed below the entity type name. A "#" indicates that an attribute is [part of] the primary identifier of the entity type, a "*"indicates the attribute is mandatory and a "○" indicates the attribute is optional. Only binary relationships are allowed, so a role corresponds to a half-line (half a relationship line). If a role is mandatory, its half-line is solid. If a role is optional, its half-line is dashed. For role cardinality, a crows-foot at the end indicates "many" and its absence indicates "1". A bar "|" across one end of a relationship indicates that the relationship is a component of the primary identifier for the entity type at that end. Part 1 discussed examples of the above notations and compared them with the corresponding ORM notations. This second article briefly discusses verbalization, then examines the Barker ER notation for exclusion constraints, frequency constraints, subtyping and non-transferable relationships.

Barker ER: verbalization

To enable the optionality and cardinality settings to be verbalized, Barker [1, p. 3-5] recommends the following *naming discipline for relationships*. Let *A R B* denote an infix relationship *R* from entity type *A* to entity type *B*. Name *R* in such a way that each of the following four patterns results in an English sentence:

each A (**must** | **may) be** *R* (**one and only one** *B* | **one or more** *B-plural-form*)

Use "must" or "may" when the first role is mandatory or optional respectively. Use "one and only one" or "one or more" when the cardinality on the second role is one or many respectively. For example, the optionality/cardinality settings in Figure 1(a) verbalize as: **each** Employee **must be** an occupier of **one and only one** Room; **each** Room **may be** occupied by **one or more** Employees.

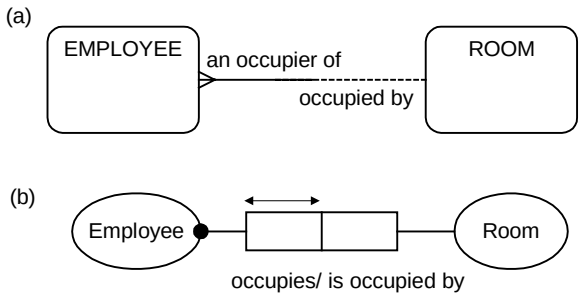

Figure 1 The ER diagram (a) is equivalent to the ORM diagram (b)

The constraints on the left hand role in the equivalent ORM model shown in Figure 1(b) verbalize as: **each** Employee occupies **some** Room; **each** Employee occupies **at most one** Room. If desired, these constraints may be combined to verbalize as: **each** Employee occupies **exactly one** Room. Since the right-hand role has no constraints, this is not normally verbalized in ORM (unlike Barker ER). However the lack of any uniqueness constraint could be verbalized explicitly as: **it is possible that the same** Room is occupied by **more than one** Employee. If no inverse reading is available, it can be verbalized as: **it is possible that more than one** Employee occupies **the same** Room. If you would like this explicit verbalization capability added as a configurable option to the verbalizer in Visio Enterprise, please email me at TerryHa@microsoft.com.

Regarding the lack of an explicit mandatory role constraint on the right-hand role, I am less inclined to want that verbalized explicitly, because it may well be unstable. If Room plays no other fact roles, the role is mandatory by implication (Room has not been declared independent), so verbalization may well confuse here. If Room does play another fact role, and we decide that some rooms may be unoccupied, we could declare this explicitly as: **it is possible that some** Room is occupied by **no** Employee. Or equivalently: **it is not necessary that each** Room is occupied by **some** Employee. If no inverse reading is available, it could be verbalized thus: **it is possible that no** Employee occupies **some** Room. If you would like an option for explicit verbalization of optional roles in Visio Enterprise, please email me your thoughts on this.

To its credit, the Barker verbalization convention is good for basic mandatory and uniqueness constraints on infix binaries. However it is far less general than ORM's approach, which applies to instances as well as types, for predicates of any arity, infix or mixfix, and covers many more kinds of constraint, with no need for pluralization. As a trivial example, the fact instance "Employee '101' an occupier of Room 23" is not proper English, but "Employee '101' occupies Room 23" is good English.

Exclusion constraints

In Barker ER notation, an exclusion constraint over two or more roles is shown as an "*exclusive arc*" connected to the roles with a small dot or circle. For example, Figure 2(a) includes the constraint that no employee may be allocated both a bus pass and a parking bay. In ORM this constraint is depicted by connecting "⊗" to the relevant roles by a dotted line, as shown in Figure 2(b).

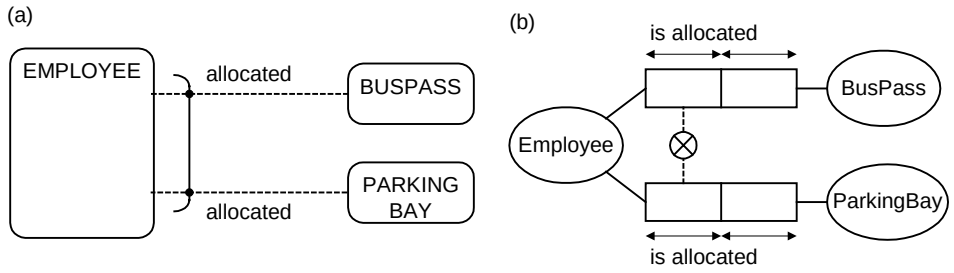

Figure 2 A simple exclusion constraint in (a) Barker ER notation and (b) ORM

To declare that two or more roles are mutually exclusive and disjunctively mandatory, the Barker notation uses the exclusive arc, but each role is shown as mandatory (solid line). For example, in Figure 3(a) each account is owned by a person or a company, but not both. This notation is liable to mislead, since it violates the orthogonality principle in language design. Viewed by itself, the first role of the association Account owned by Person would appear to be mandatory, since a solid line is used. But the role is actually optional, since superimposing the exclusive arc changes the semantics of the solid line to mean the role belongs to a set of roles that are disjunctively mandatory.

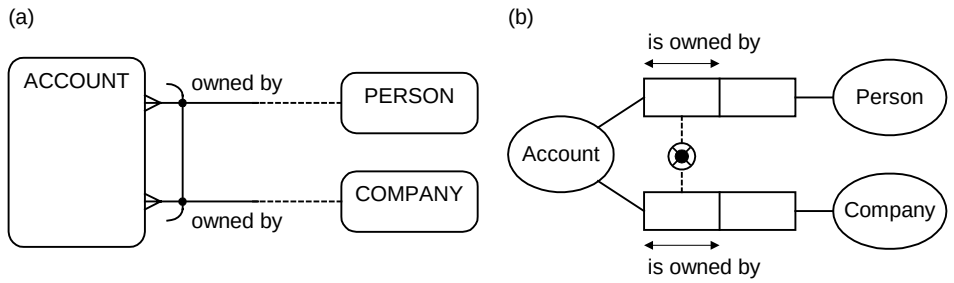

Figure 3 An exclusive-or constraint in (a) Barker ER notation and (b) ORM

Contrast this with the equivalent ORM model shown in Figure 3(b). Here an exclusion constraint ⊗ is orthogonally combined with a disjunctive mandatory (inclusive-or) constraint ⊙ to produce an exclusive-or constraint, shown here by the "lifebuoy" or partition symbol formed by overlaying one constraint symbol on the other. As an alternative, the inclusive-or and exclusion constraints may be displayed separately.

The ORM notation makes it clear that each role is individually optional, and that the exclusive-or constraint is a combination of inclusive-or and exclusion constraints. Suppose we modified our business so that the same account could be owned by both a person and a company. Removing just the exclusion constraint from the model leaves us with the inclusive-or constraint ⊙ that each account is owned by a person or company. Like UML, the Barker ER notation doesn't even have a symbol for an inclusive-or constraint, so is unable to diagram this or the many other cases of this nature that occur in practice.

In the Barker notation, a role may occur in at most one exclusive arc. ORM has no such restriction. For example, in Figure 4(a) no student can be both ethnic and aboriginal, and no student can be both an aboriginal and a migrant (these rules come from a student record system in Australia). Even if Barker notation supported unaries (it doesn't) this situation could not be handled by exclusive arcs. Like UML, Barker ER does not provide a graphic notation for exclusion constraints over role-sequences. For instance, it cannot capture the ORM pair-exclusion constraint in Figure 4(b), which declares that no person who wrote a book may review the same book. Such rules are very common. Moreover, the Barker notation cannot express any ORM subset or equality constraints at all, even over simple roles.

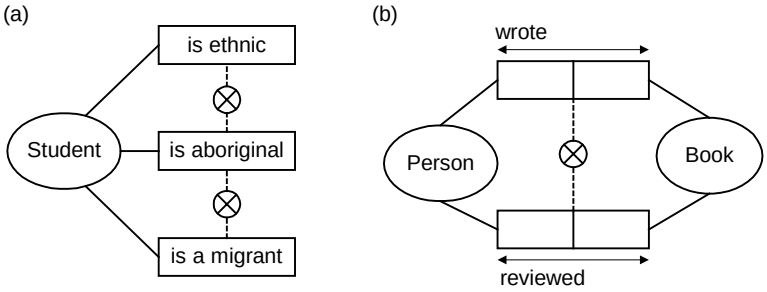

Figure 4 Some ORM exclusion constraints not handled by exclusive arcs in Barker ER notation

Frequency constraints

The Barker ER notation allows simple frequency constraints to be specified. For any positive integer *n*, a constraint of the form $= n$, $< n$, $\leq n$, $> n$, $\geq n$ may be written beside a single role to indicate the number of instances that may be associated with an instance playing the other role. For example, the frequency constraint "≤ 2" in Figure 5 indicates that each person is a child of at most two parents. In the Barker notation, this constraint is placed on the parent role, making it easy to read the constraint as a sentence starting at the other role. In ORM the constraint is placed on the child role, making it easy to see the impact of the constraint on the population (each person appears at most twice in the child role population). Unlike the Barker notation, ORM allows frequency constraints to include ranges (e.g. 2-5) and to apply to role-sequences.

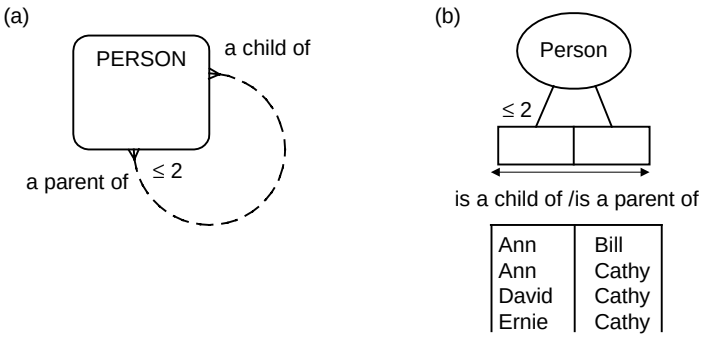

Figure 5 A simple frequency constraint in (a) Barker ER notation and (b) ORM

Subtyping

In Barker ER notation, subtyping is depicted with a version of Euler diagrams. In effect, only partitions (exclusive and exhaustive) can be displayed. For example, Figure 6(a) indicates that each patient is a male patient or female patent but not both. Like UML, ORM displays subtyping using directed acyclic graphs (DAGs). Subtype exclusion and exhaustion constraints are normally omitted in ORM, as in Figure 6(b), since they are implied by the subtype definition and other constraints (e.g. mandatory, uniqueness and value constraints on Patient is of Gender). However they can be explicitly displayed as in Figure 6(b).

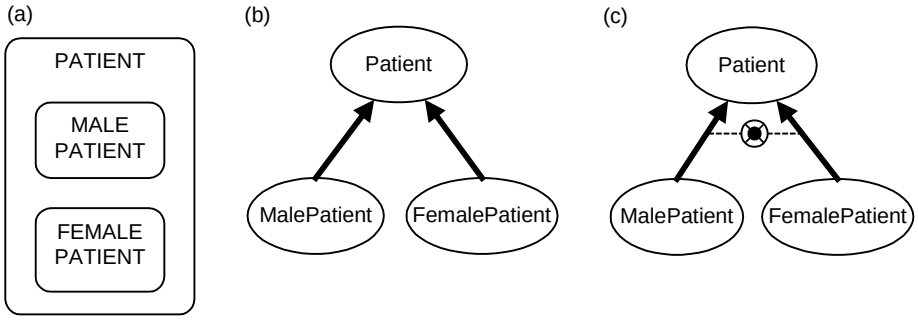

Figure 6 A subtype partition in (a) Barker ER, (b) implicit ORM and (c) explicit ORM notation

Euler diagrams are good for simple cases, since they intuitively show the subtype inside its supertype. However unlike DAGs, they are hopeless for complex cases (e.g. many overlapping subtypes), and they make it inconvenient to attach details to the subtypes. For the latter reason, attributes are often omitted from subtypes when the Barker notation is used.

In the Barker notation, if the original subtype list is not exhaustive, an "Other" subtype is added to make it so, even if it plays no specific role. For example, in Figure 7 a vehicle is a car or truck or possibly something else, and a car is a sedan or wagon or possible something else.

A major problem with the Barker notation for subtyping is that it does not depict overlapping subtypes (e.g. Manager and FemaleEmployee as subtypes of Employee) or multiple inheritance (e.g. FemaleManager as a subtype of FemaleEmployee and Manager). While it is possible to implement multiple inheritance in single inheritance systems (e.g. Java) by using some low level tricks, for conceptual modeling purposes multiple inheritance should be simply modeled as multiple inheritance.

As a final comparison point about subtyping, Barker ER lacks ORM's capability for formal subtype definitions and context-dependent identification schemes.

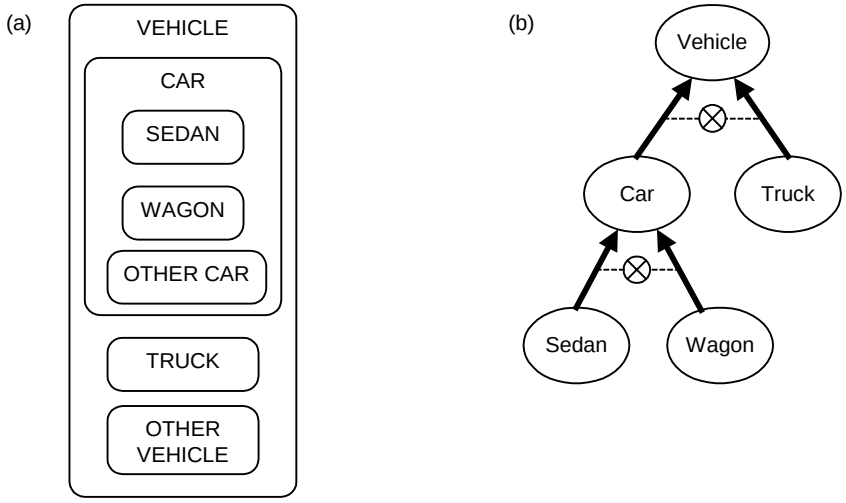

Figure 7 Non-exhaustive, exclusive subtypes in (a) Barker ER and (b) ORM

Non-transferable relationships

In addition to its static constraint notation, Barker ER includes a dynamic "changeability constraint" for marking "*non-transferable relationships*". This constraint declares that once an instance of an entity type plays a role with an object, it cannot ever play this role with another object. This is indicated by adding an open diamond to the constrained role. For example, Figure 8(a) declares that the birth country of a person is non-transferable.

As indicated in Figure 8(b), ORM does not currently include a notation for this constraint. It would be possible to add a notation for this (as well as UML's changeability settings of changeable, frozen, addOnly), but it is at least debatable whether this is advisable. If we were to add such a notation, we would need to ensure that the implemented model is still open to error corrections by duly authorized users. For example, if my birth country was mistakenly entered as Austria, it should be possible to change this to Australia. For further discussion on this issue, see my comments on changeability properties in [2]. If you have some strong views on this issue, please email me your thoughts.

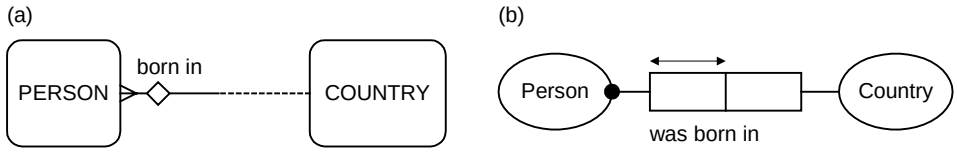

Figure 8 Non-transferable nature of a relationship is declared in (a) Barker ER, but not in (b) ORM

Conclusion

The Barker ER notation does a good job of expressing simple mandatory, uniqueness, exclusion and frequency constraints, simple subtyping and also non-transferable relationships. However, if a feature is modeled as an attribute instead of as a relationship, very few of these constraints can be specified for it. In contrast to ORM, the Barker ER notation does not support unary, *n*-ary or objectified associations (nesting). Moreover it lacks support for most of the advanced ORM constraints (e.g. subset, multi-role exclusion, ring constraints and join constraints). It does not include a formal textual language such as ConQuer for specifying queries, other constraints and derivation rules at the conceptual level. Nevertheless it is better than many other notations for ER modeling, and is still widely used. If you ever need to specify a model in Barker ER notation, I suggest you first do the model in ORM, then map it to the Barker notation and make a note of any rules that can't be expressed there diagrammatically.

Next issues

Later articles in this series will examine the Information Engineering notation for ER, before concluding with a discussion of IDEF1X.

References

1. Barker, R. 1990, *CASE*Method: Tasks and Deliverables*, Addison-Wesley, Wokingham, England.
2. Halpin, T.A. 1999, 'UML data models from an ORM perspective: Part 10', *Journal of Conceptual Modeling*, InConcept, Minneapolis USA.

Entity Relationship modeling from an ORM perspective: Part 3

Terry Halpin
Microsoft Corporation

Introduction

This article is the third in a series of articles dealing with Entity Relationship (ER) modeling from the perspective of Object Role Modeling (ORM). Part 1 provided a brief overview of the ER approach, and then covered the basics of the Barker ER notation. Part 2 completed the examination of the Barker ER notation by discussing verbalization, exclusion constraints, frequency constraints, subtyping and non-transferable relationships. Both parts compared the Barker notations with the corresponding ORM notations This article discusses the Information Engineering notation for ER, relating it to relevant ORM constructs.

The *Information Engineering* (IE) approach began with the work of Clive Finkelstein in Australia, and CACI in the UK, and was later adapted by James Martin. Different versions of IE exist, with no single standard. In one form or other, IE is supported by many data modeling tools, and is one of the most popular notations for database design.

Entity types, attributes and associations

In the IE approach, *entity types* are shown as named rectangles, as in Figure 1(a). A*ttributes* are often displayed in a compartment below the entity type name, as in Figure 1(b), but are sometimes displayed separately (e.g. bubble charts). Some versions support basic constraints on attributes (e.g. Ma/Op/Unique).

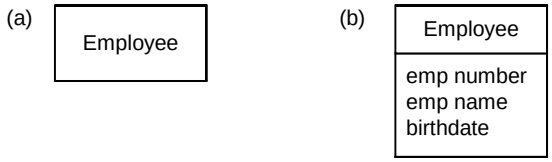

Figure 1 Typical IE notation for (a) entity type and (b) entity type with attributes

Relationships are typically restricted to binary associations only, which are shown as named lines connecting the entity types. As with the Barker notation, a half-line or line end corresponds to a role in ORM. Optionality and cardinality settings are indicated by annotating the line ends. To indicate that a role is *optional*, a circle "○" is placed at the other end of the line, signifying a minimum participation frequency of 0. To indicate that a role is *mandatory*, a stroke "|" is placed at the other end of the line, signifying a minimum participation frequency of 1. After experimenting with some different notations for a cardinality of "*many*", Finkelstein settled on the intuitive crow's foot symbol suggested by Dr. Gordon Everest.

In conjunction with a minimum frequency of 0 or 1, a stroke "|" is often used to indicate a maximum frequency of 1. With this arrangement, the combination "○|" indicates "*at most one*" and the combination "||" indicates "*exactly one*". This is the convention that I will use in this section. However different IE conventions exist. For example, some assume a maximum cardinality of 1 if no crows foot is used, and hence use just a single "|" for "exactly one". Clive Finkelstein uses the combination "○|" to mean "optional but will become mandatory", which is really a dynamic rather than static constraint—this can be combined with a crow's foot. Some conventions allow a crow's foot to mean the minimum (and hence maximum) frequency is many. So if you are using a version of IE, you should check which of these conventions applies.

Figure 2 shows a simple IE diagram and its equivalent ORM diagram. With IE, as you read an association from left to right, you verbalize the constraint symbols at the right-hand end. Here, each employee occupies exactly one (at least 1 and at most 1) room. Although inverse predicates are not always supported in IE, you can supply these yourself to obtain a verbalization in the other direction. For example:

"Each room is occupied by zero or more employees". As with the Barker notation, a plural form of the entity type name is introduced to deal with the many case.

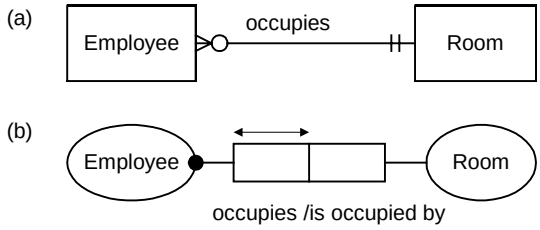

Figure 2 The IE diagram (a) is equivalent to the ORM diagram (b)

The IE notation is similar to the Barker notation in showing the maximum frequency of a role by marking the role at the other end. But unlike the Barker notation, the IE notation shows the optionality/mandatory setting at the other end as well. In this sense, IE is like UML (even though different symbols are used). As discussed in an earlier article, there are sixteen possible constraint patterns for optionality and cardinality on binary associations. Figure 3 shows eight cases in IE notation together with the equivalent cases in ORM.

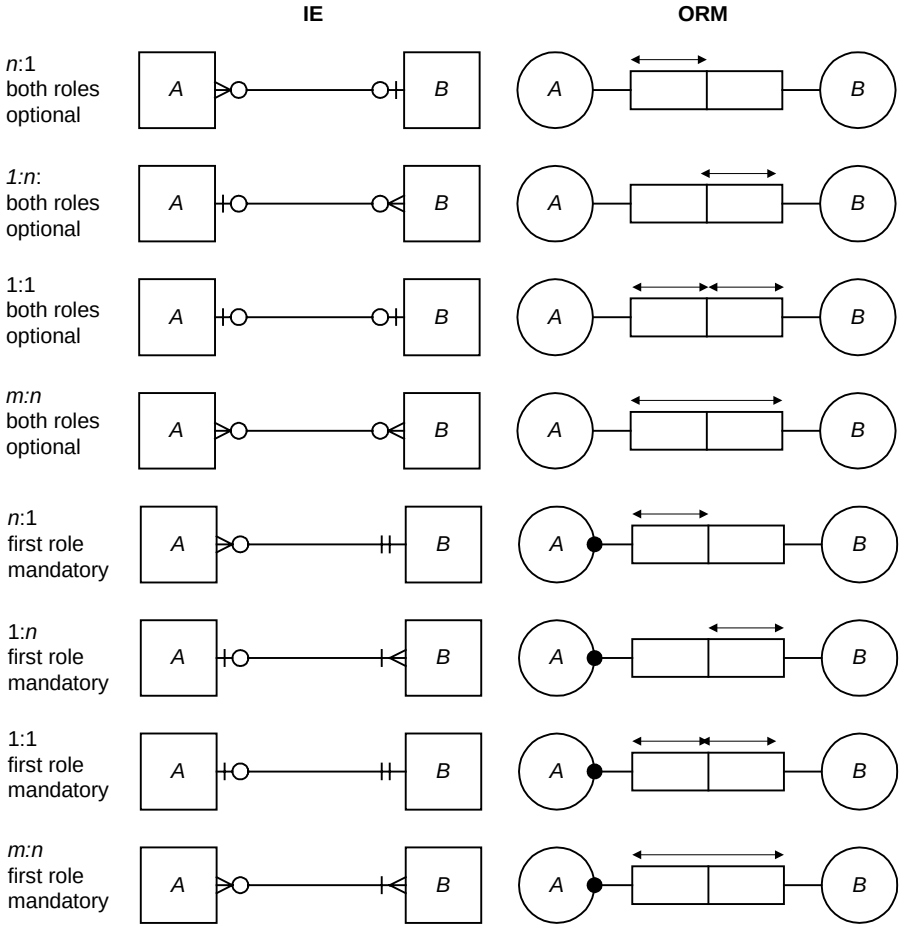

Figure 3 Some equivalent constraint patterns

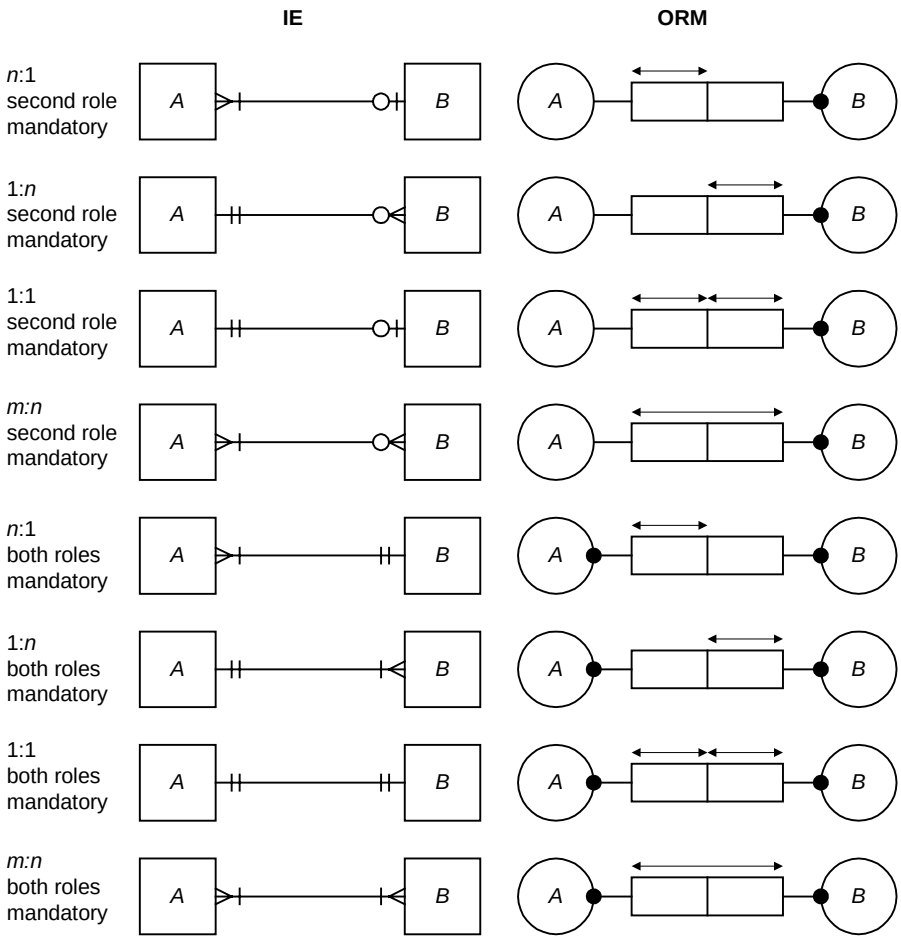

Figure 4 Other equivalent cases

The other eight cases are shown in Figure 4. An example using the different notation for IE used by Finkelstein is shown in Figure 5. Here the single bar on the left end of the association indicates that each computer is located in exactly one office. The circle, bar and crow's foot on the right end of the association collectively indicate that each office must eventually house one or more computers. This "optional becoming mandatory" constraint has no counterpart in ORM, and is not supported in most IE modeling tools.

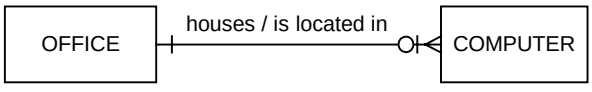

Figure 5 Finkelstein's notation for IE is different from ours

Some modeling tools support the IE notation for *n*:1, 1:*n* and 1:1 associations but not *m:n* (many to many) associations. For such tools, four of the sixteen cases cannot be directly represented. In this situation, you can model the *m:n* cases indirectly by introducing an "*intersection entity type*" with mandatory *n*:1 associations to the original entity types. For example, the *m:n* case with both roles optional may be handled by introducing the object type *C* as shown in Figure 6 for both IE and ORM. In ORM, *C* is a *co-referenced* object type, and the transformation is an instance of a flatten/coreference equivalence.

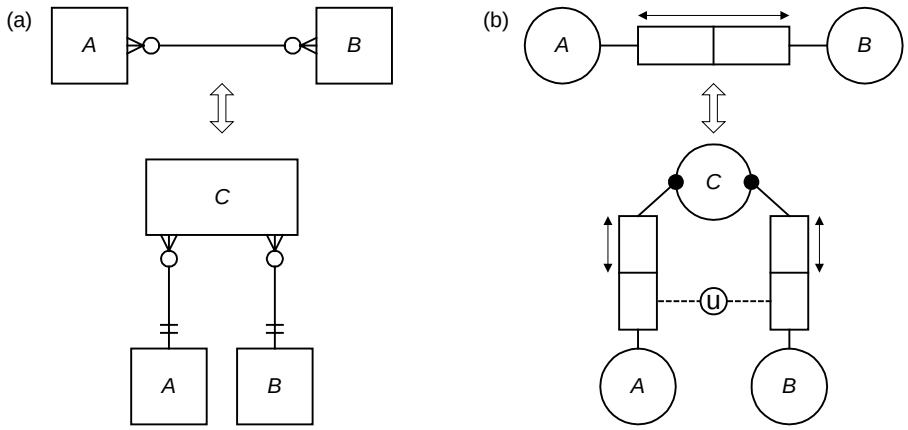

Figure 6 An *m:n* association remodeled as an entity type with *n*:1 associations

As an example, the *m:n* association Person plays Sport can be transformed into the mandatory *n*:1 associations: Play is by Person; Play is of Sport. However such a transformation is often very unnatural, especially if nothing else is recorded about the co-referenced object type. So any truly conceptual approach must allow *m:n* associations to be modeled directly.

Advanced constraints and subtyping

Some versions of IE support an *exclusive-or constraint*, shown as a black dot connected to the alternatives. Figure 7(a) depicts the situation where each employee is allocated a bus pass or parking bay, but not both. The equivalent ORM schema is shown in Figure 7(b). Unlike ORM, IE does not support an inclusive-or constraint. Nor does it support exclusion constraints over multi-role sequences.

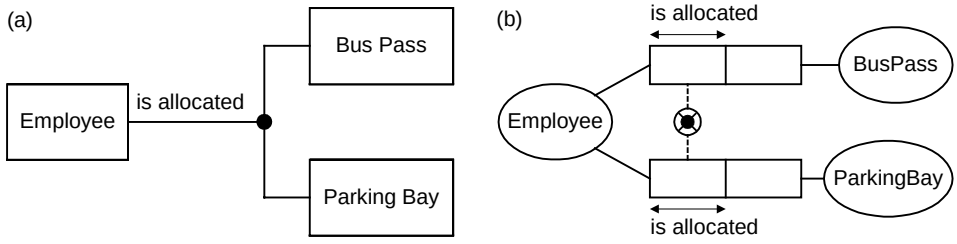

Figure 7 An exclusive-or constraint in (a) IE and (b) ORM

Subtyping schemes for IE vary. Sometimes Euler diagrams are used, adding a blank compartment if needed for "Other". Sometimes directed acyclic graphs are used, possibly including subtype relationship names and optionality/cardinality constraints. Figure 8 show three different subtyping notations for partitioning Patient into the subtypes MalePatient and FemalePatient. There is no formal support for subtype definitions, and no provision for context-dependent reference. Multiple inheritance may or may not be supported, depending on the version.

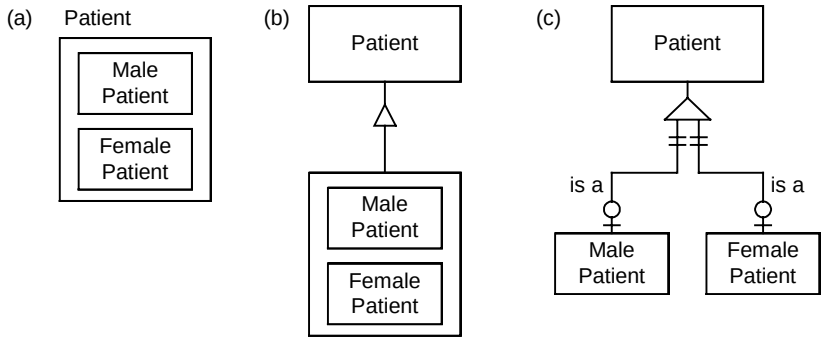

Figure 8 Some different IE notations for subtyping

Conclusion

Although far less expressive than ORM, IE does a good job of covering basic constraints. If you ever need to specify a model in IE notation, I suggest you first do the model in ORM, then map it to IE and make a note of any rules that can't be expressed there diagrammatically. Often referred to as "the father of information engineering", Clive Finkelstein is an amiable Aussie who is still actively engaged in the information engineering discipline. He developed a set of modeling procedures to go with the notation, extended IE to Enterprise Engineering (EE), and recently began applying data modeling to work in XML (Extensible Markup Language). An overview of IE by Clive can be found in [1], and details on his recent work are given in [2]. For a look at the IE approach used by James Martin, see [3]. Martin's more recent books tend to use the UML notation instead. In practice however, IE is used far more extensively for database design than is UML, which is mostly used for object-oriented code design.

Next issue

The next article in this series discusses the IDEF1X notation.

References

1. Finkelstein, C. 1998, 'Information engineering methodology', *Handbook on Architectures of Information Systems*, eds. P. Bernus, K. Mertins & G. Schmidt, Springer-Verlag, Berlin, Germany, pp. 405-427.
2. Finkelstein, C. & Aiken, P. 2000, *Building Corporate Portals with XML*, McGraw-Hill, New York.
3. Martin, J. 1993, *Principles of Object Oriented Analysis and Design*, Prentice Hall, Englewood Cliffs, NJ.